Semantics beyond Circuits

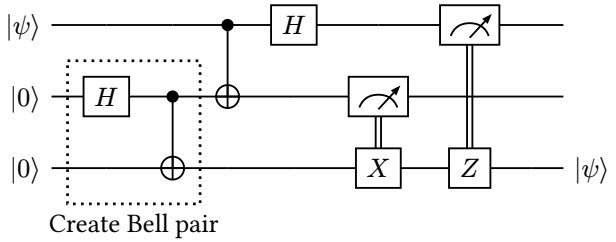
Quantum Circuits

Quantum circuits blocks of straight-line code

Example


```
fn bellPair() {
  let a, b = new Qubit()
  H a
  CX a b
  return (a, b)
}

fn teleport(q) {
  let a, b = bellPair()
  CX q a
  H q

  let qres = measRec q
  let ares = measRec a

  if (ares) { X b}
  if (qres) { Z b }

  return b
}
```

(a) Code



(b) Circuit

Figure 1: Quantum teleportation, 2 ways

Quantum Circuits

Circuits are foundational in QC, but not terribly expressive

Quantum Circuits

Circuits are foundational in QC, but not terribly expressive

$\rightarrow$ straight-line code

Quantum Circuits

Circuits are foundational in QC, but not terribly expressive

$\rightarrow$ straight-line code

$\rightarrow$ No loops

Quantum Circuits

Circuits are foundational in QC, but not terribly expressive

$\rightarrow$ straight-line code

$\rightarrow$ No loops

$\rightarrow$ Not turing complete

hierarchies

Classical computing observes an *expressiveness hierarchy* based on the level of control flow offered:

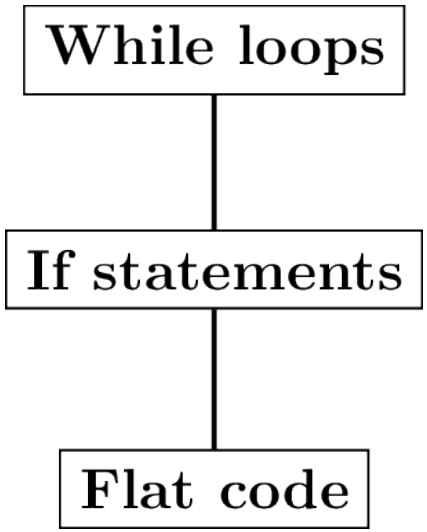


Figure 2: Classical CF Lattice

hierarchies

There is an analogous hierarchy in QC, though it looks more complicated

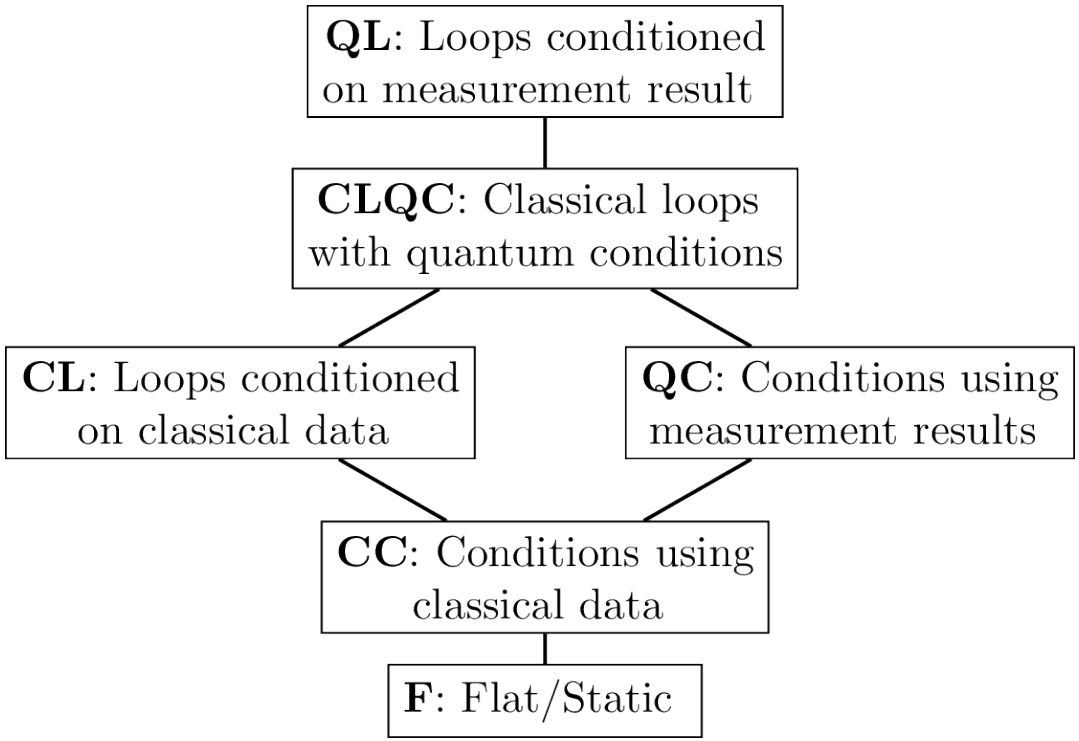


Figure 3: CF Lattice for QC

Why the complication?

Measurement

We tend to assume QC follow the *co-processor architecture*

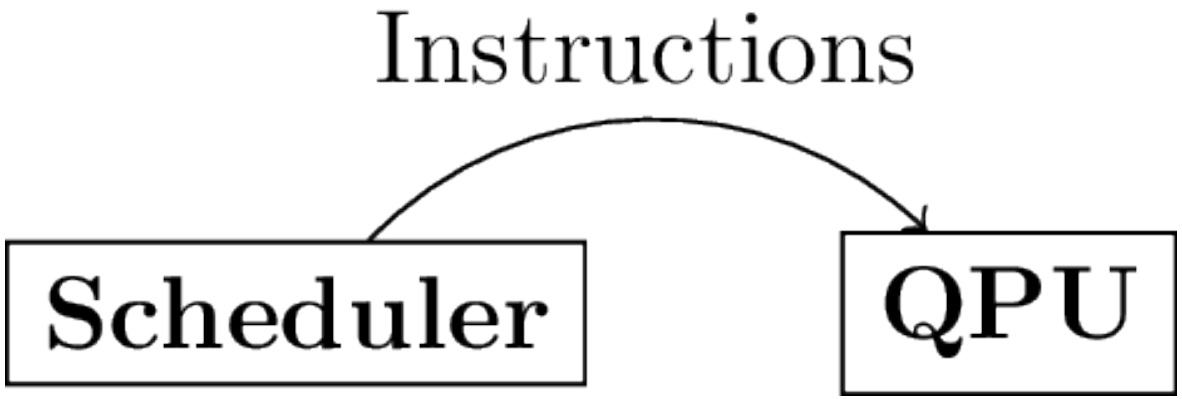


Figure 4: Co-processor Architecture

Measurement

We tend to assume QC follow the *co-processor architecture*

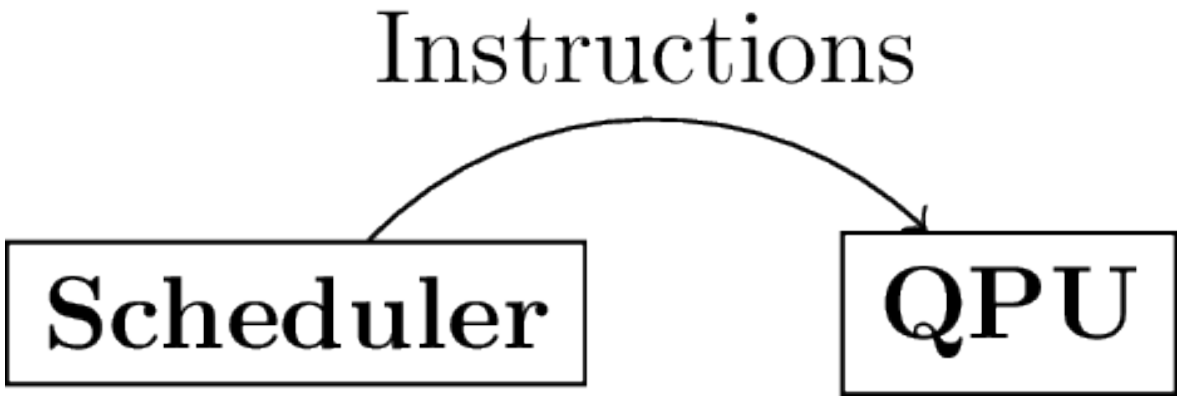


Figure 4: Co-processor Architecture

- *QPU* is a quantum computer

Measurement

We tend to assume QC follow the *co-processor architecture*

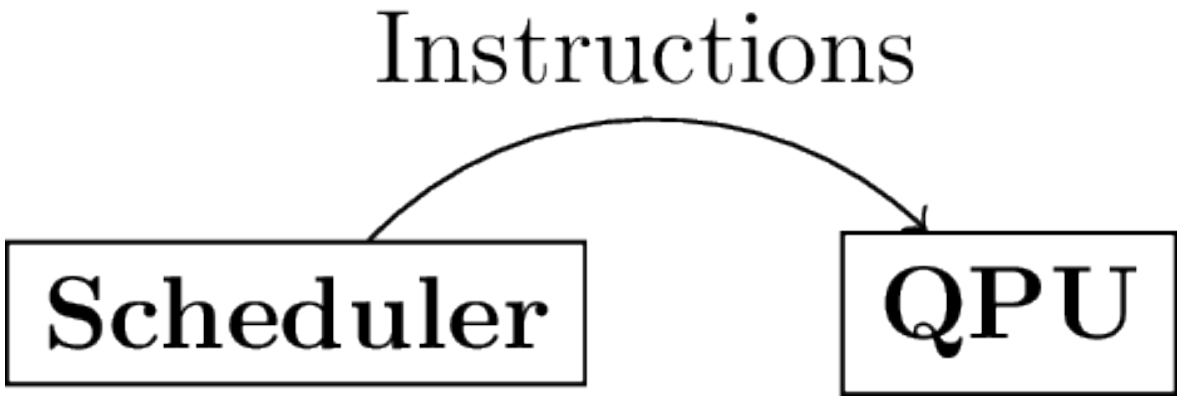


Figure 4: Co-processor Architecture

- *QPU* is a quantum computer
- *scheduler* is just a classical computer that tells the QPU what to do

Measurement

Adding measurement adds a backwards edge to this graph:

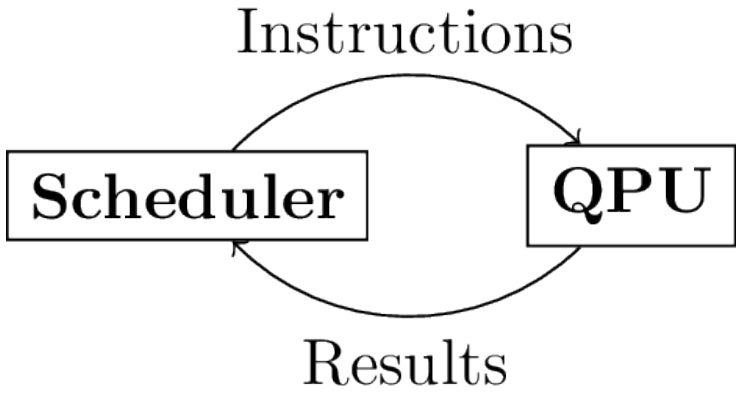


Figure 5: Co-processor Architecture with Dynamic Lifting

Measurement

Adding measurement adds a backwards edge to this graph:

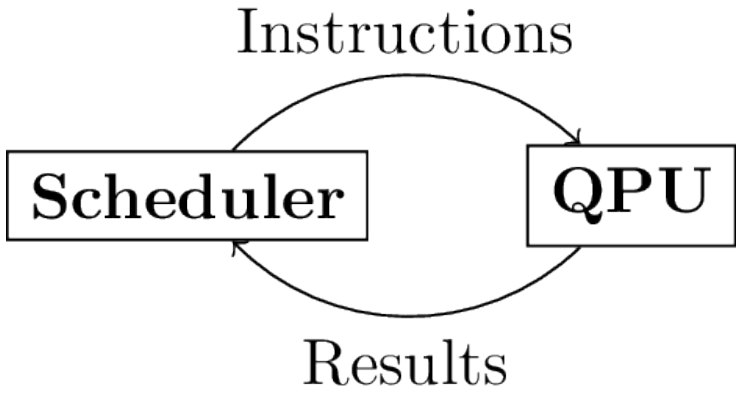


Figure 5: Co-processor Architecture with Dynamic Lifting

- Passing results back from the QPU to the scheduler is called *dynamic lifting*

Measurement

Adding measurement adds a backwards edge to this graph:

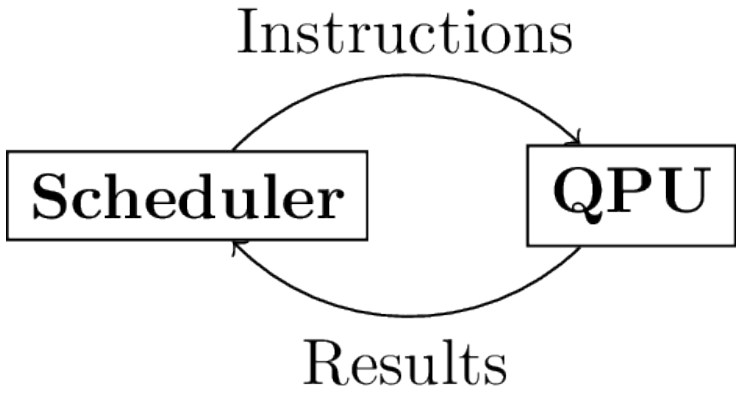


Figure 5: Co-processor Architecture with Dynamic Lifting

- Passing results back from the QPU to the scheduler is called *dynamic lifting*
- And generally, it's considered expensive

What I'm working on

Circuits represent QC and below

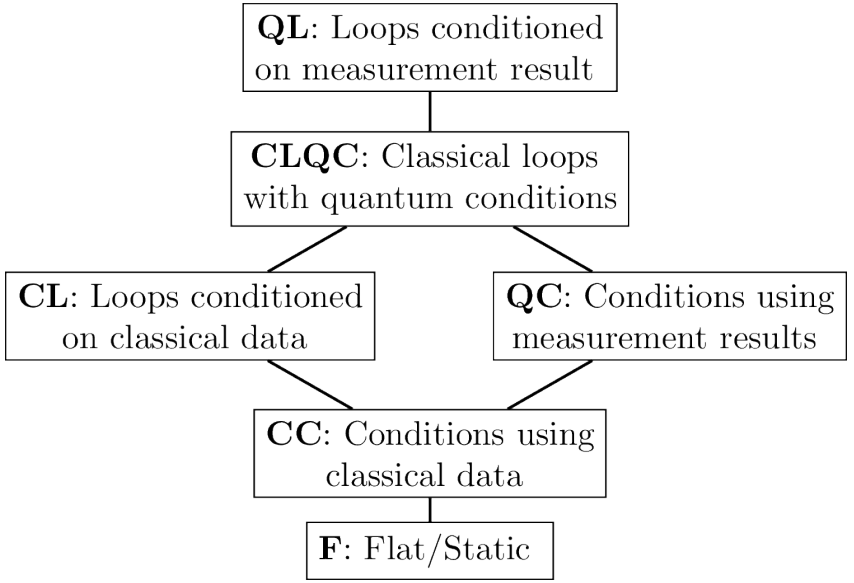


Figure 6: CF Lattice for QC

What I'm working on

Circuits represent QC and below

Semantics of circuits well understood

But higher levels of control flow are more complicated

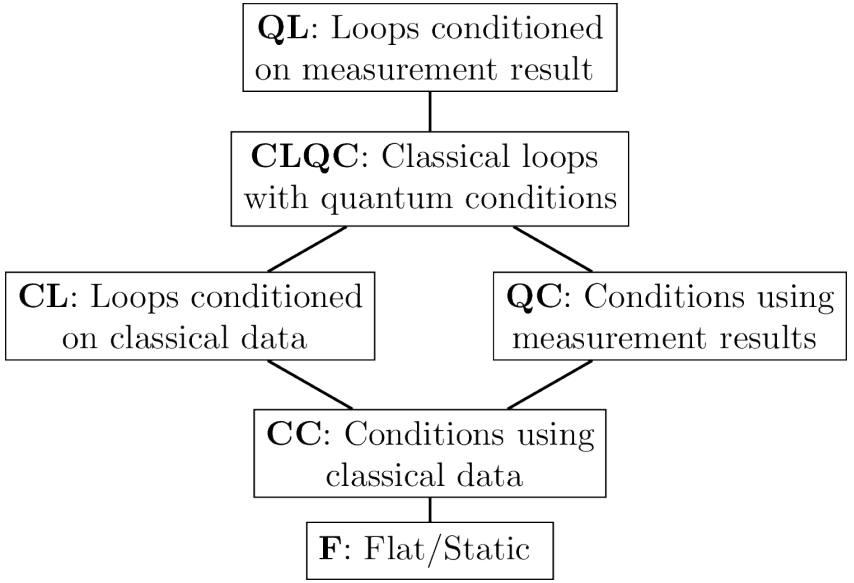


Figure 6: CF Lattice for QC

What I'm working on

The big idea:

- A family of languages

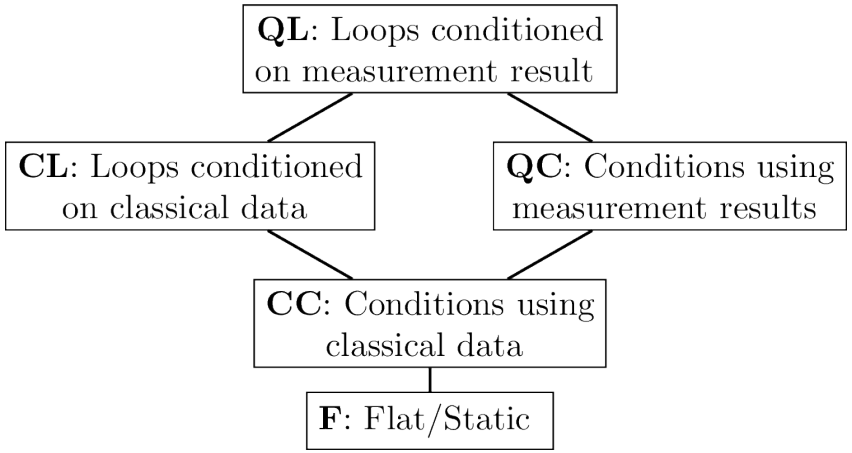


Figure 7: (Reduced) CF Lattice for QC

What I'm working on

The big idea:

- A family of languages
- One for each level in this Lattice

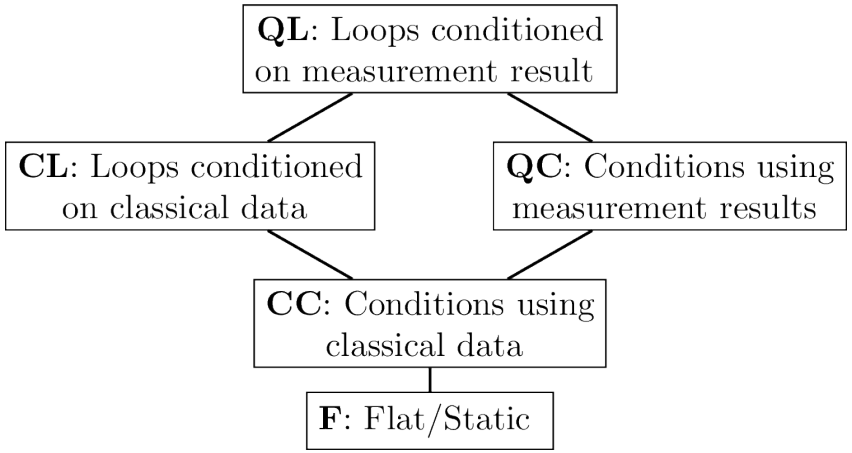


Figure 9: (Reduced) CF Lattice for QC

What I'm working on

The big idea:

- A family of languages
- One for each level in this Lattice
- Related by simple addition of syntactic features

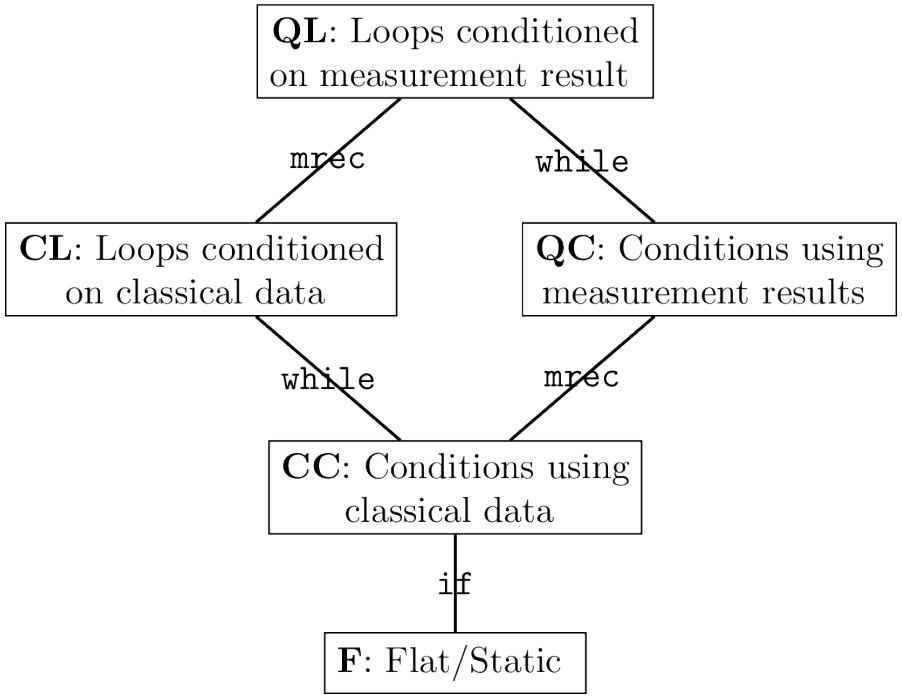


Figure 9: (Reduced) CF Lattice for QC with syntactic feature labels

What I'm working on

The big idea:

- A family of languages
- One for each level in this Lattice
- Related by simple addition of syntactic features

→ give a semantics to each, and we have a nice framework for studying

- languages
- algorithms
- hardware

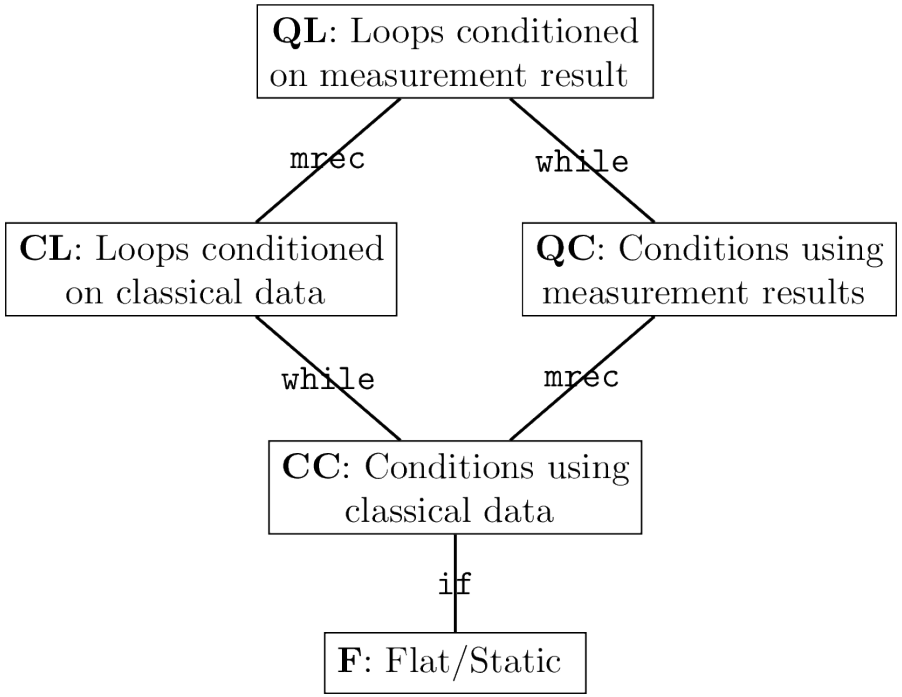


Figure 9: (Reduced) CF Lattice for QC with syntactic feature labels

Where are we now?

SQIR is a mechanically (i.e. in ROQC) specified quantum intermediate language

Part of the larger VOQC project (verified optimisation)

Could be a good jumping off point for this work...

I'll either:

- extend SQIR
- or start from scratch (if so, probably in lean)
  - for higher levels in the lattice, this might be easier, since SQIR has no handling for classical data